



BEACH SIMULATOR

Mapa de inserções de game design

Onde cada mecânica pode nascer, como os módulos podem conversar e quais limites mantêm o código simples para trabalho humano.

BASE DO CÓDIGO	Arquitetura local validada em 21 de julho de 2026
BASE DE DESIGN	Jesse Schell - The Art of Game Design: A Book of Lenses
REGRA CENTRAL	Adicionar regras onde elas pertencem. Criar um novo script somente quando existir uma responsabilidade nova e duradoura.

1. Como usar este mapa

Este documento não é uma autorização para criar qualquer feature em qualquer lugar. Ele é um mapa de decisão: primeiro encontre a natureza da mecânica, depois altere o módulo que já possui aquele estado ou regra.

Regra de bom senso

- Se a feature muda uma regra existente, modifique o modelo que já controla essa regra.
- Se muda apenas a aparência, altere a view ou o módulo visual correspondente.
- Se agenda algo para o futuro, use a timeline de spawn ou o lifecycle do dia.
- Se cruza dois domínios, retorne um resultado pequeno e encaminhe-o no ponto de composição.
- Só crie outro arquivo quando o conceito tiver nome próprio, estado próprio e expectativa real de crescimento.

Sinais de overengineering

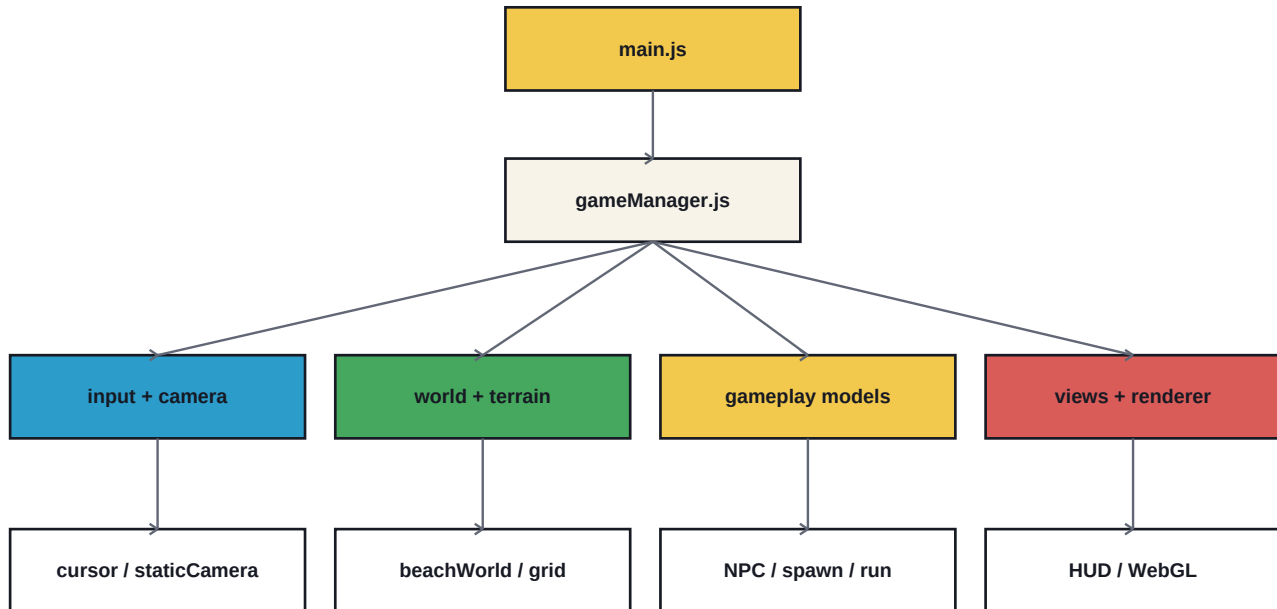
- Um arquivo novo contém apenas uma chamada para outro arquivo.
- É necessário abrir quatro scripts para entender um clique.
- Um Manager genérico conhece câmera, UI, NPCs e dinheiro ao mesmo tempo.
- Eventos são criados para chamadas que poderiam ser uma função direta e local.
- DTOs, registries e factories existem sem mais de uma implementação real.

Leitura das lentes

As lentes citadas abaixo foram usadas como perguntas de design, não como padrões de software. A lente explica por que uma mecânica pode melhorar a experiência; o módulo indicado explica onde implementá-la sem contaminar a arquitetura.

2. Arquitetura resultante

A direção permanece de cima para baixo. **gameManager.js** compõe os sistemas e mantém o loop. Os módulos inferiores não o importam. A montagem concreta da praia agora vive em **world/beachWorld.js**, reduzindo o GameManager sem criar um framework de orquestração.



Interpretação

- main.js apenas inicia o jogo.
- gameManager.js conecta input, loop, modelos e apresentação.
- beachWorld.js monta terrain, cenário, grid, população inicial e assets.
- Modelos de gameplay não importam DOM, câmera ou renderer.
- Views recebem snapshots ou comandos de apresentação; não decidem economia ou comportamento.

3. Mapa rápido

Use esta tabela para localizar rapidamente o ponto mais provável de uma alteração.

SCRIPT	DECISÃO DE DESIGN	EXEMPLO
world/beachWorld.js	Composição inicial do mapa	Adicionar construção estática, posição inicial ou reserva de área
terrain/beachGrid.js	Espaço jogável e ocupação	Colisão, footprint de loja, células bloqueadas
terrain/terrainWorld.js	Forma e visual do terreno	Areia molhada, restinga, densidade de palmeiras
spawn/spawnManager.js	Quando algo acontece	Chegada de banhista, tubarão, lixo ou tesouro
spawn/spawnableObjectDto.js	O que pode existir	Novo lixo, animal, valioso ou obstáculo
npcs/npcSystem.js	O que um NPC decide	Necessidades, conversa, nado, fuga e saída
buildings/buildingServicesModel.js	Efeito e custo de serviço	Itens, preços, salário e manutenção
interaction/cleanBeachController.js	Consequência de coletar	Dinheiro, progresso, ferramenta e combo
onboarding/cleanBeachGuide.js	Ajuda contextual da primeira task	Atraso, escolha do alvo e prioridade visual da seta
run/dayLifecycleController.js	Ritmo do dia	Clímax, fechamento, cobrança e abertura de spawn
camera/staticCamera.js	Estado e movimento da câmera	Foco, limites e dados para minimapa
input/cursor.js	Gestos brutos	Touch, drag, zoom e rotação
ui/*.js	Como o estado é mostrado	HUD, relatório, tooltip, aviso e animação

4. Mundo, terreno e espaço

world/beachWorld.js	
RESPONSABILIDADE ATUAL	Carrega assets e monta a praia inicial: terrain, kiosk, beach house, grid, NPCs vazios e cinco objetos de coleta.
INSERÇÕES DE GAME DESIGN	• Balancear em planInitialPopulation a mistura dos cinco objetos iniciais e o valioso garantido. • Posicionar novos edifícios que já existam no começo da run. • Reservar footprints para impedir objetos e NPCs de nascerem dentro de construções. • Definir a composição visual inicial que produz o primeiro impacto da run.
NÓ DE COMUNICAÇÃO	planInitialPopulation retorna pedidos DTO; loadBeachWorld retorna referências concretas e resolveWorldObjectPosition ao GameManager.
NÃO COLOCAR AQUI	Preço, manutenção, comportamento de NPC, temporizadores ou regras de recompensa.
BASE NO LIVRO	Lente #21 Espaço Funcional (p. 135); #62 Interesse Inerente (p. 254); #72 Controle Indireto (p. 293).

terrain/beachGrid.js	
RESPONSABILIDADE ATUAL	Divide o mapa em mar, areia e restinga; converte posições em células e registra ocupação.
INSERÇÕES DE GAME DESIGN	• Footprints de prédios com tamanhos diferentes. • Consulta isWalkable para colisão simples dos banhistas. • Custos de navegação: areia seca, areia molhada e água rasa.
NÓ DE COMUNICAÇÃO	NPC e placement podem consultar o grid por uma função injetada. O grid não deve conhecer NPCs nem construções específicas.
NÃO COLOCAR AQUI	Animação, dinheiro, spawn por tempo ou acesso ao renderer.
BASE NO LIVRO	Lente #21 Espaço Funcional (p. 135); #22 Estado Dinâmico (p. 140); #42 Simplicidade/Complexidade (p. 196).

terrain/terrainWorld.js	
RESPONSABILIDADE ATUAL	Constrói e atualiza as células visuais de areia, restinga, mar e palmeiras ao redor da câmera.
INSERÇÕES DE GAME DESIGN	• Faixa de areia molhada ou variação visual por zona. • Distribuição, escala e espaçamento de palmeiras. • Decorações puramente visuais e janela infinita do terreno.
NÓ DE COMUNICAÇÃO	Pode receber estado visual simples, como intensidade de maré, sem importar TimeManager.
NÃO COLOCAR AQUI	Colisão, preço de terreno, necessidades ou regras de spawn.
BASE NO LIVRO	Lente #7 Tétrade Elemental (p. 43); #63 Beleza (p. 255); #72 Controle Indireto (p. 293).

5. Spawn e objetos

spawn/spawnManager.js

RESPONSABILIDADE ATUAL	Agenda eventos, controla intervalos de chegada e transforma banhistas em fontes futuras de lixo.
INSERÇÕES DE GAME DESIGN	• Novos canais: tubarão, caranguejo, água-viva, rush de clientes ou tesouro. • Probabilidade, cooldown, prioridade e custo de atenção de cada evento. • Pacing adaptativo usando rating e, futuramente, um resumo do DNA do jogador.
NÓ DE COMUNICAÇÃO	Retorna pedidos de spawn. Não instancia modelos 3D e não altera HUD ou economia diretamente.
NÃO COLOCAR AQUI	Posição WebGL, animação, cobrança ou diálogo de NPC.
BASE NO LIVRO	Lente #2 Surpresa (p. 26); #29 Chance (p. 169); #39 Tempo (p. 189); #61 Curva de Interesse (p. 252).

spawn/scheduledSpawnQueue.js

RESPONSABILIDADE ATUAL	Mantém a fila temporal ordenada dos eventos futuros.
INSERÇÕES DE GAME DESIGN	• Somente critérios técnicos de desempate necessários ao agendamento. • Suporte a cancelamento de um evento quando a condição de gameplay desaparece.
NÓ DE COMUNICAÇÃO	Recebe DTOs de evento do SpawnManager e devolve o próximo evento. Deve permanecer ignorante sobre praia, NPC ou tubarão.
NÃO COLOCAR AQUI	Probabilidades, narrativa, recompensas ou escolha do tipo de evento.
BASE NO LIVRO	Lente #39 Tempo (p. 189); #43 Elegância (p. 198).

spawn/spawnableObjectDto.js

RESPONSABILIDADE ATUAL	Catálogo universal dos tipos, categorias, zonas, fontes e traits dos objetos que podem existir.
INSERÇÕES DE GAME DESIGN	• Cadastrar coco, joia, pedra, tronco, animal ou novo lixo. • Declarar traits reutilizáveis como dirty, pickup, obstacle ou predator. • Definir em quais zonas e fontes o objeto é permitido.
NÓ DE COMUNICAÇÃO	Outros módulos consultam o DTO por type. O DTO descreve; ele não executa comportamento.
NÃO COLOCAR AQUI	Tempo de spawn, preço, animação ou efeitos concretos sobre NPCs.
BASE NO LIVRO	Lente #22 Estado Dinâmico (p. 140); #23 Emergência (p. 143); #26 Regras (p. 150).

6. NPCs e construções

npcs/npcSystem.js	
RESPONSABILIDADE ATUAL	Mantém entidades, movimento, aceleração, atividades, necessidades, reclamações e saída da praia.
INSERÇÕES DE GAME DESIGN	• Interação entre dois banhistas: conversa, amizade, discussão ou grupo familiar. • Nome, idade, perfil de higiene, tolerância ao calor e avaliação final. • Estados de nadar, fugir de animal, comprar bebida e procurar banheiro.
NÓ DE COMUNICAÇÃO	update pode retornar eventos pequenos, como departures e interactions. O GameManager encaminha esses resultados para rating, economia ou UI.
NÃO COLOCAR AQUI	DOM, balões visuais, carregamento de GLTF, dinheiro ou agendamento global.
BASE NO LIVRO	Lente #16 Jogador (p. 106); #19 Necessidades (p. 127); #23 Emergência (p. 143); #44 Personagem (p. 199).

npcs/npcWorld.js	
RESPONSABILIDADE ATUAL	Carrega o modelo PicoCAD/GLTF e converte snapshots dos NPCs em corpo e pernas renderizáveis.
INSERÇÕES DE GAME DESIGN	• Novas animações de nadar, reclamar, comprar e conversar. • Variações visuais de bather mantendo o mesmo comportamento. • Offsets explícitos de orientação para cada novo modelo.
NÓ DE COMUNICAÇÃO	Recebe snapshots prontos do NpcSystem. Não decide para onde ou por que o NPC anda.
NÃO COLOCAR AQUI	Necessidades, pathfinding, avaliação, preços ou regras de saída.
BASE NO LIVRO	Lente #7 Tétrade Elemental (p. 43); #44 Personagem (p. 199); #57 Feedback (p. 230).

6. NPCs e construções - serviços

buildings/buildingServicesModel.js	
RESPONSABILIDADE ATUAL	Registra construções possuídas e calcula seus efeitos, manutenção e custos ao fim do dia.
INSERÇÕES DE GAME DESIGN	• Itens e preços da Beverage Store enquanto o catálogo ainda for pequeno. • Estoque, margem, salário de lifeguard e custo de manutenção do banheiro. • Trade-offs claros: renda versus lixo, segurança versus salário, higiene versus manutenção.
NÓ DE COMUNICAÇÃO	Recebe comandos como addBuilding ou purchase. Retorna resultado financeiro; a economia registra a transação.
NÃO COLOCAR AQUI	Modelos 3D, modal de escolha, movimento de NPC ou ordem temporal dos eventos.
BASE NO LIVRO	Lente #32 Escolhas Significativas (p. 181); #33 Triangularidade (p. 182); #46 Economia (p. 204); #47 Balanceamento (p. 205).

scenery/sceneryWorld.js	
RESPONSABILIDADE ATUAL	Define assets, escala, orientação e instâncias visuais de kiosk, beach house e placeholders.
INSERÇÕES DE GAME DESIGN	• Cadastrar o asset visual de toilet, trash can, Wi-Fi spot e beverage store. • Corrigir yaw, tamanho e brilho específicos de cada modelo.
NÓ DE COMUNICAÇÃO	Recebe type e position definidos pela composição do mundo ou pela escolha de construção.
NÃO COLOCAR AQUI	Preço, efeito de serviço, manutenção ou escolha aleatória da run.
BASE NO LIVRO	Lente #7 Tétrade Elemental (p. 43); #63 Beleza (p. 255).

7. Interação, input e câmera

interaction/cleanBeachController.js

RESPONSABILIDADE ATUAL	Aplica as consequências de uma coleta: dinheiro, experiência e avanço da task.
INSERÇÕES DE GAME DESIGN	• Valores diferentes para moeda, anel e joia. • Ferramenta que permite remover um hazard específico. • Combo ou bônus por sequência rápida de limpeza, se o playtest justificar.
NÓ DE COMUNICAÇÃO	Recebe o objeto coletado e chama somente os modelos envolvidos. Retorna um resultado simples para feedback visual.
NÃO COLOCAR AQUI	Hit test, DOM, animação, câmera ou criação de objetos.
BASE NO LIVRO	Lente #40 Recompensa (p. 191); #49 Progresso Visível (p. 214); #57 Feedback (p. 230); #58 Juiciness (p. 233).

interaction/worldObjectInteraction.js

RESPONSABILIDADE ATUAL	Projeta objetos 3D no canvas ou no HUD e resolve hover ou clique usando área de trigger em pixels.
INSERÇÕES DE GAME DESIGN	• Seleção por caixa, drag de construção ou prioridade entre objetos sobrepostos. • Ajuste de hit area por categoria para acessibilidade. • Posicionamento de novos balões ou indicadores ligados a entidades do mundo.
NÓ DE COMUNICAÇÃO	Retorna seleção ou coordenadas projetadas. Um controller de gameplay decide o efeito do clique.
NÃO COLOCAR AQUI	Dinheiro, task, remoção do inventário ou regras de NPC.
BASE NO LIVRO	Lente #48 Acessibilidade (p. 213); #53 Controle (p. 222); #55 Interface Virtual (p. 226).

input/cursor.js

RESPONSABILIDADE ATUAL	Normaliza mouse e teclado em pan, rotação, zoom, seleção e edge scrolling.
INSERÇÕES DE GAME DESIGN	• Mapeamento mobile de pinch, toque, drag com um dedo e rotação com dois dedos. • Configuração de sensibilidade e inversão de direção.
NÓ DE COMUNICAÇÃO	Emite intenções de input. Nunca chama NPC, terrain ou GameManager diretamente.
NÃO COLOCAR AQUI	Recompensa, colisão, spawn, câmera concreta ou manipulação do mundo.
BASE NO LIVRO	Lente #35 Cabeça e Mãos (p. 185); #53 Controle (p. 222); #54 Interface Física (p. 226).

camera/staticCamera.js

RESPONSABILIDADE ATUAL	Mantém alvo, distância, yaw, pan, rotação e zoom da câmera RTS.
INSERÇÕES DE GAME DESIGN	• Foco em um NPC, construção ou evento urgente. • Limites de navegação e retorno suave ao gameplay. • Fornecer target e bounds para um futuro minimapa.
NÓ DE COMUNICAÇÃO	Para minimapa, o ponto de composição lê camera.getTarget e entrega um snapshot a uma MinimapView. A câmera não importa a view.
NÃO COLOCAR AQUI	DOM, ícones de minimapa, regras do dia, spawn ou dinheiro.
BASE NO LIVRO	Lente #53 Controle (p. 222); #56 Transparência (p. 227); #64 Projeção (p. 257).

8. Run, tempo e métricas

run/dayLifecycleController.js

RESPONSABILIDADE ATUAL	Inicia o dia, libera spawns, encerra o dia, para a timeline e registra custos de serviços.
INSERÇÕES DE GAME DESIGN	• Disparar o clímax dos últimos quarenta segundos. • Preparar o resumo final e transicionar ao próximo dia. • Aplicar modificadores de run antes de abrir os spawns.
NÓ DE COMUNICAÇÃO	Coordena modelos do ciclo do dia, mas não monta UI nem decide comportamento individual de NPC.
NÃO COLOCAR AQUI	Renderização, câmera, asset, diálogo ou regras internas de uma construção.
BASE NO LIVRO	Lente #39 Tempo (p. 189) ; #60 Modos (p. 240) ; #61 Curva de Interesse (p. 252) .

run/runSessionModel.js

RESPONSABILIDADE ATUAL	Guarda dia, total de dias e fase atual da run.
INSERÇÕES DE GAME DESIGN	• Progressão Day 1 até Day 5. • Estados day-intro, active, day-complete e run-complete. • Condições de vitória ou derrota da run quando forem aprovadas.
NÓ DE COMUNICAÇÃO	O lifecycle solicita transições; views observam snapshots encaminhados pelo ponto de composição.
NÃO COLOCAR AQUI	Contagem regressiva por frame, UI, cobrança ou spawn.
BASE NO LIVRO	Lente #25 Objetivos (p. 149) ; #39 Tempo (p. 189) ; #60 Modos (p. 240) .

time/timeManager.js

RESPONSABILIDADE ATUAL	Controla duração, tempo restante, start e update do relógio do dia.
INSERÇÕES DE GAME DESIGN	• Duração diferente por dificuldade ou modificador de run. • Pause explícito durante modal, se o design decidir manter essa regra. • Sinal de janela final para o clímax do dia.
NÓ DE COMUNICAÇÃO	Retorna snapshot temporal. Outros módulos escolhem como reagir ao tempo.
NÃO COLOCAR AQUI	Spawn concreto, salário, task, animação ou texto do relógio.
BASE NO LIVRO	Lente #18 Flow (p. 122) ; #39 Tempo (p. 189) ; #61 Curva de Interesse (p. 252) .

economy/beachEconomyModel.js

RESPONSABILIDADE ATUAL	Mantém saldo e histórico mínimo de entradas e despesas em centavos.
INSERÇÕES DE GAME DESIGN	• Categorias de transação para o relatório do dia. • Limites de caixa ou dívida apenas se virem regra aprovada.
NÓ DE COMUNICAÇÃO	Serviços e controllers enviam transações com sourceld. A economia não pergunta por que a compra ocorreu.
NÃO COLOCAR AQUI	Preço de item, probabilidade de venda, UI ou comportamento de cliente.
BASE NO LIVRO	Lente #5 Valor Endógeno (p. 32) ; #40 Recompensa (p. 191) ; #46 Economia (p. 204) .

8. Run, tempo e métricas - avaliação

ratings/beachRatingModel.js	
RESPONSABILIDADE ATUAL	Armazena avaliações e calcula média para HUD e frequência de spawn.
INSERÇÕES DE GAME DESIGN	• Avaliação na saída do bather baseada em limpeza, necessidades e segurança. • Peso ou recência das avaliações após playtests. • Resumo de motivos positivos e negativos no fechamento do dia.
NÓ DE COMUNICAÇÃO	NpcSystem retorna departures; um controller converte a experiência final em rating e chama submitRating.
NÃO COLOCAR AQUI	Mover NPC, mostrar estrelas, alterar spawn diretamente ou decidir animação.
BASE NO LIVRO	Lente #20 Julgamento (p. 128); #40 Recompensa (p. 191); #57 Feedback (p. 230).

experience/playerExperienceModel.js	
RESPONSABILIDADE ATUAL	Acumula sinais de domínio, pressão, exploração, fadiga e necessidade de orientação.
INSERÇÕES DE GAME DESIGN	• Ajustar somente pacing, hints ou seleção de eventos futuros. • Detectar jogador perdido sem punir ou reduzir recompensas silenciosamente.
NÓ DE COMUNICAÇÃO	Expõe um snapshot resumido ao SpawnManager ou onboarding. Não recebe DOM nem controla diretamente a timeline.
NÃO COLOCAR AQUI	Decidir resultados financeiros, adulterar rating ou substituir regras claras por dificuldade invisível.
BASE NO LIVRO	Lente #16 Jogador (p. 106); #18 Flow (p. 122); #31 Desafio (p. 179).

9. UI, feedback e renderização

ui/buildingChoice.js

RESPONSABILIDADE ATUAL	Mantém a escolha roguelike atual e apresenta três opções ao jogador.
INSERÇÕES DE GAME DESIGN	• Mostrar vantagem, custo diário e risco de cada construção. • Garantir que a consequência escolhida apareça ainda no mesmo minuto. • Variar opções por run sem criar uma estratégia dominante.
NÓ DE COMUNICAÇÃO	A view devolve apenas o type escolhido. BuildingServices aplica a regra; scenery cria o visual.
NÃO COLOCAR AQUI	Cobrar dinheiro, spawnar NPC ou alterar terrain diretamente.
BASE NO LIVRO	Lente #32 Escolhas Significativas (p. 181); #33 Triangularidade (p. 182); #40 Recompensa (p. 191).

ui/taskList.js

RESPONSABILIDADE ATUAL	Mantém e apresenta tasks, prioridade, dependências e progresso.
INSERÇÕES DE GAME DESIGN	• Objetivos concretos de curto prazo com contador e barra. • Tasks condicionais para limpeza, satisfação ou segurança. • Revelar a próxima task somente quando o jogador tiver contexto.
NÓ DE COMUNICAÇÃO	Controllers avançam tasks por id. A view reage ao snapshot e não consulta o mundo.
NÃO COLOCAR AQUI	Remover objeto, pagar recompensa ou decidir spawn.
BASE NO LIVRO	Lente #25 Objetivos (p. 149); #49 Progresso Visível (p. 214); #57 Feedback (p. 230).

onboarding/cleanBeachGuide.js + views de feedback

RESPONSABILIDADE ATUAL	O guia escolhe um item pendente após inatividade; OnboardingView e PickupFeedbackView apresentam mão, seta, avisos, +1 e \$5.
INSERÇÕES DE GAME DESIGN	• Ajustar a janela de 3 a 5 segundos usando resultados de playtest. • Priorizar objetos visíveis, valiosos ou distantes conforme a intenção do tutorial. • Adicionar uma segunda dica contextual sem colocar a regra dentro da view.
NÓ DE COMUNICAÇÃO	CleanBeachGuide consulta task e objetos, escolhe o alvo e envia coordenadas projetadas. As views apenas desenharam o feedback.
NÃO COLOCAR AQUI	Alterar dinheiro, remover objeto, concluir task ou controlar câmera.
BASE NO LIVRO	Lente #48 Acessibilidade (p. 213); #57 Feedback (p. 230); #58 Juiciness (p. 233).

rendering/worldRenderer.js

RESPONSABILIDADE ATUAL	Executa WebGL, shaders, sprites, iluminação e frame final em resolução interna fixa.
INSERÇÕES DE GAME DESIGN	• Shader PSX, jittering, água, transparência, fog e otimizações gráficas. • Novos passes visuais apenas quando um efeito não cabe no material atual.
NÓ DE COMUNICAÇÃO	Recebe sceneObjects prontos. Não consulta GameManager nem modelos de domínio.
NÃO COLOCAR AQUI	Rating, spawn, seleção, preço, colisão ou comportamento de NPC.
BASE NO LIVRO	Lente #7 Tétrade Elemental (p. 43); #63 Beleza (p. 255); #64 Projeção (p. 257).

10. Três exemplos de inserção

A. Loja com itens e preços

- buildings/buildingServicesModel.js: lista pequena de bebidas, preço, margem e estoque.
- npcs/npcSystem.js: estado BUYING e decisão de procurar a loja.
- economy/beachEconomyModel.js: somente registra a venda já calculada.
- scenery/sceneryWorld.js: asset e orientação visual da loja.
- ui/buildingChoice.js: comunica o trade-off antes da escolha.

B. Dois banhistas interagem

- npcs/npcSystem.js detecta proximidade e cria um interaction result.
- O mesmo sistema altera os estados dos dois NPCs para TALKING ou ARGUING.
- npcs/npcWorld.js anima os dois corpos.
- Uma view recebe uma fala somente quando existir texto para apresentar.
- SpawnManager não participa: a interação nasce do estado atual, não de um relógio global.

C. Abrir um nó para minimapa

- camera/staticCamera.js já fornece target e orientação.
- terrain/beachGrid.js fornece zonas e limites lógicos.
- NPCSystem e world objects fornecem snapshots mínimos de posição.
- Uma futura MinimapView recebe um objeto simples com esses dados.
- GameManager apenas reúne o snapshot durante render; câmera, NPC e grid não importam a view.

Contrato sugerido, apenas quando o minimapa for aprovado

```
minimapView.render({ cameraTarget, cameraYaw, zones, npcs, objects })
```

11. Backlog recomendado

Ordem sugerida para aumentar a experiência sem voltar a concentrar regras no GameManager.

SCRIPT	DECISÃO DE DESIGN	EXEMPLO
1. Cinco dias	Fechar o loop principal	advanceDay, reset do relógio e run-complete
2. Colisão de NPC	Preservar legibilidade espacial	isWalkable do beachGrid injetado no NpcSystem
3. Construções significativas	Fazer escolhas importarem	Trade-offs visíveis e consequência no mesmo minuto
4. Primeiro cliente	Criar vínculo e curiosidade	Nome, necessidade e primeira avaliação
5. Rating completo	Fechar feedback sistêmico	Departure gera avaliação e altera frequência futura
6. Clímax e relatório	Dar forma aos cinco minutos	Evento final, saldo, rating, limpeza e teaser
7. Eventos emergentes	Aumentar variedade	Interações de NPC, animal, lixo e tesouro

Critério para criar um novo script

Crie o arquivo apenas quando pelo menos duas destas condições forem verdadeiras: há estado próprio; há regra própria; há mais de um consumidor; o conceito possui testes próprios; o arquivo atual deixaria de ser compreensível com a nova feature. Caso contrário, modifique o método ou módulo existente.

Lentes prioritárias para os próximos cinco minutos

- #25 Objetivos: o jogador sempre sabe o que fazer agora.
- #32 Escolhas Significativas: cada prédio muda a run.
- #39 Tempo: eventos cabem no ritmo de cinco minutos.
- #49 Progresso Visível: limpeza e satisfação têm evidência concreta.
- #57 Feedback e #58 Juiciness: cada ação responde imediatamente.
- #61 Curva de Interesse: gancho, descanso, escalada, clímax e resolução.

12. Material de referência

SHELL, Jesse. *The Art of Game Design: A Book of Lenses*. Arquivo local consultado em material/Jesse Schell - The Art of Game Design A Book of Lenses (1).pdf.

Como o livro foi usado

O documento não copia soluções prontas do livro. As lentes foram usadas para avaliar intenção, feedback, ritmo, escolha, espaço e experiência do jogador. As decisões de arquitetura foram derivadas do código local e das regras operacionais do projeto.

Lentes citadas

SCRIPT	DECISÃO DE DESIGN	EXEMPLO
#2 / #4	Surpresa e Curiosidade	Criar perguntas e variação sem confusão
#16 / #19	Jogador e Necessidades	Projetar bathers legíveis e humanos
#21 / #22 / #23	Espaço, Estado e Emergência	Mundo sistêmico sem acoplamento
#25 / #26	Objetivos e Regras	Tasks concretas e contratos claros
#32 / #33	Escolhas e Triangularidade	Trade-offs de construções e risco
#39 / #40 / #46	Tempo, Recompensa e Economia	Run curta com valor compreensível
#42 / #43	Simplicidade e Elegância	Poucos conceitos, cada um com função clara
#48 / #49	Acessibilidade e Progresso	Input tolerante e objetivos visíveis
#53 a #58	Controle, Interface e Feedback	Ação responsiva sem lógica no DOM
#61 / #72	Curva e Controle Indireto	Ritmo dirigido sem retirar liberdade

Documento vivo: atualize este mapa quando uma feature alterar a responsabilidade real de um módulo. Não atualize o código para obedecer ao documento; atualize o documento para refletir um código simples e verdadeiro.